History-dependent returns and a shallower 215-state ZFC machine

Exact transition certificates, a three-command cache calculus, and semantic refutation alarms

Research contribution for the ProveIt repository

21 October 2026

Abstract

We give two explicit deterministic, two-symbol, blank-tape machines with 215 working states, including initialization, and an additional terminal state. The first changes five transitions of the pinned accepted K215 machine. Six short, context-dependent blocks prove its blank halting equivalence by positive-duration stuttering. The useful observation is that the preceding one or two transitions have already written the marker bits needed to choose an existing return path. This releases two rows from the register-engine obligation without adding a state.

The second machine combines those returns with a different decoder allocation and a closed source reassociation. It has maximum source-address depth 46 rather than 47, changes 1,042 primitive addresses, and has a transition graph that is not a relabelling of the accepted table. Both machines pass fresh quantified runtime derivations for all 6,176 guarded primitive cases and independent complete blank-prefix checks. Using the repository's inherited numerical-calculus calibration, ordinary reasoning formalizable in ZFC gives $\neg\,\mathrm{Halt}(M, \mathsf{blank}) \leftrightarrow \mathrm{Con}(\mathsf{ZFC})$ for each machine.

On the calculus side, we give a complete proof compiler for a three-command transfer/cache design with 201 distinct source subtrees, and a sound extension of the falsity alarm to every membership-atom code. These are separate source-level results. No 214-state machine is claimed. The remaining allocation obstruction and the precise scope of the negative searches are recorded so that the next reduction can build on concrete evidence.

Contents

1 Results, conventions, and the pinned starting point **3**

2 Using transition history as an already available marker **3**

2.1 The five literal row changes . . . 3

2.2 The exact incoming-context lemmas . . . 4

2.3 Six complete local blocks . . . 4

2.4 What the engine change buys . . . 5

3 A different 215-state decoder **6**

3.1 One exact source rewrite . . . 6

3.2 The frozen physical table . . . 6

4 Complete blank starts and quantified binary runtime **7**

5 The resulting consistency equivalences **8**

6 A 201-subtree source: transfer-pop and cache-return inference **8**

6.1 Numerical calculus and exact command interface . . . 9

6.2 Literal construction and eight variants . . . 10

6.3 Soundness on all raw candidate words . . . 10

6.4 Structural blocks recovered without DROP . . . 10

6.5 The strong theorem-block compiler . . . 11

6.6 Cofinal successful candidates and the source theorem . . . 12

7 A semantic alarm: every membership atom detects inconsistency **13**

7.1 A general replacement principle . . . 13

7.2 All raw codes of residue one . . . 14

7.3 Exact scanner and measured cost . . . 14

8 What the searches establish, and what remains open **15**

8.1 Source size and physical sharing are different objectives . . . 15

8.2 Finite search boundaries . . . 15

8.3 The next concrete targets . . . 16

9 Reproduction and incorporation **17**

A Complete dense transition table for the shallower 215-state machine **19**

1 Results, conventions, and the pinned starting point

The starting point is the repository's accepted `Computability/BusyBeaver/Research/ZFC/candidate-k215` at commit

`75716498437bfc7291d5900cab2983a11fd54bac`.

Its ordinary proof identifies blank halting with a finite ZFC refutation [1]. The exact native table has SHA-256

`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`.

This input theorem and the inherited numerical calibration are credited inputs, not new results of this packet. The new transition proofs, source reassociation, exact tables, runtime runs, and source-calculus compiler composition are supplied here.

Construction	TM states	Source subtrees	Leaves	Depth
Pinned K_{215}	215	202	4,751	47
History-return K_{215}^{hist}	215	202	4,751	47
Shallower decoder $K_{215}^{\mathrm{shallow}}$	215	202	4,751	46
Direct transfer/cache source	—	201	4,709	44
Membership-alarm source	—	206	4,746	47

Table 1: Accepted physical counts and separately measured source counts. A dash means that no accepted physical machine of the target size is supplied for that source. Source subtrees include distinct primitive leaves.

The machine model is a two-way infinite tape indexed by $\mathbb{Z}$, with alphabet $\{0,1\}$, initially all zero. A row $(q,b) \mapsto (w,d,z)$ writes w, moves one cell in direction $d \in \{L,R\}$, and enters z. Native start is 0 and native halt is 254. All 215 working labels have both read rows; halt has none. The nonconsecutive native labels are inherited names, not extra states. Dense versions use working labels $0,\dots,214$ and halt 215. Initialization is inside each counted table; there is no externally supplied program tape or uncounted preprocessing.

The source consists of a full binary tree whose leaves are increments CI_j, guarded decrements CD_j, structured breaks CB_k, structured continues CC_k, and halt CH. The nine logical registers have order (N,T,W,A,D,H,X,Y,L). A source-subtree count is a measure of shared program syntax. It is not the binary state count: physical states can share decoder and register-engine duties, and some decoder roles depend on a known preceding tape bit.

The principal files are `machine/k215-shallow.native.tm` and `machine/k215-shallow.tm`. The simpler bridge is `machine/k215-context.native.tm`. Its small delta makes the new engine argument particularly easy to audit. The source tables, complete compressed runtime cases, exact startup traces, independent checkers, mathematical notes, and inherited notices are included. A manifest pins every file.

2 Using transition history as an already available marker

2.1 The five literal row changes

Define K_{215}^{hist} by changing exactly the following five rows of K_{215}. All labels in the proof below are native labels.

State	Read	Old write/move/target	New write/move/target
13	1	$1, L, 31$	$1, L, 81$
27	1	$0, L, 40$	$0, L, 302$
83	1	$0, L, 42$	$0, L, 57$
156	1	$1, R, 176$	$1, R, 52$
198	0	$1, L, 40$	$1, L, 54$

The new native table has SHA-256

`da20914ad9bda3f75de0381f2c05a7b5e67d26199b57b870a5ea3b72511ab627.`

No label is added or removed. All 430 rows remain defined, and all working states are reachable in the underlying directed transition graph.

The argument does not assert that these two tables act alike from every arbitrary configuration. Their modified entries are equivalent on contexts forced by actual incoming transitions. That is sufficient for runs from the fixed start state, and it exposes sharing opportunities that a context-free state minimizer misses.

2.2 The exact incoming-context lemmas

Write a_i for the tape symbol i cells from the head at entry. Every entry to state 27 is through $95/0 = 0R27$, so $a_{-1} = 0$. Every entry to state 83 is through $73/1 = 1R83$, so $a_{-1} = 1$. Every entry to state 156 is through $189/1 = 1L156$, so $a_1 = 1$. Every entry to state 198 is through $66/0 = 1R198$, so $a_{-1} = 1$. These statements hold in both tables. None of those entry states is the start state.

The state-13 entry needs a two-transition suffix. Its possible incoming edges are $8/1 = 1R13$, $38/0 = 0R13$, and $42/0 = 0R13$. Enumerating all two-edge incoming histories in each literal table establishes

$$q = 13 \ \wedge \ a_0 = 1 \quad \Longrightarrow \quad a_{-1} = 1 \ \vee \ (a_{-1} = 0 \wedge a_{-2} = 1). \tag{1}$$

The finite checker retains the latest write at each position while walking the two edges backwards. It deliberately includes some read-incompatible histories: this is an overapproximation, so checking the displayed property on all retained histories is sound. An actual two-step suffix is always represented. State 13 is neither the initial state nor a possible one-transition successor of it, so there is no unhandled short-prefix exception.

This use of history costs no machine state. The marker is already on the tape because the incoming edge wrote it. The proof, rather than a new runtime flag, carries the knowledge of that marker.

2.3 Six complete local blocks

The table lists the exact state paths. The entry bit and context columns give every cell that the corresponding trace reads. No other tape cell is inspected, so arbitrary contents outside the listed cells are allowed.

Entry	Context	Old state path	New state path	Final head
13/1	$a_{-1} = 0, a_{-2} = 1$	$13 \to 31 \to 21$	$13 \to 81 \to 111 \to 128 \to 21$	0
13/1	$a_{-1} = 1$	$13 \to 31 \to 47 \to 16$	$13 \to 81 \to 57 \to 16$	-1
27/1	$a_{-1} = 0$	$27 \to 40 \to 4$	$27 \to 302 \to 4$	-2
83/1	$a_{-1} = 1$	$83 \to 42 \to 16$	$83 \to 57 \to 16$	-2
156/1	$a_1 = 1$	$156 \to 176 \to 150$	$156 \to 52 \to 150$	2
198/0	$a_{-1} = 1$	$198 \to 40 \to 54$	$198 \to 54 \to 54$	-2

In the first case both tapes are unchanged; the new trace takes four transitions instead of two. In the second case both erase the entry cell and leave the predecessor one unchanged. The third case erases the entry one. The fourth erases both adjacent ones. The fifth changes no tape cell. The sixth changes the entry zero to one. In every case the final state, head, and entire tape are identical. Every strict interior transition of every listed block is outside the set of five modified rows.

Theorem 2.1 (Exact history-return equivalence)**.** *From the fixed blank start,*

$$\mathrm{Halt}(K_{215}^{\mathrm{hist}}, \mathsf{blank}) \quad \Longleftrightarrow \quad \mathrm{Halt}(K_{215}, \mathsf{blank}).$$

The equivalence follows from a finite, positive-duration stuttering simulation and is formalizable in ZFC.

Proof. Couple the two runs at identical configurations. If their next row is unmodified, take the same single transition. Otherwise the incoming-context lemma selects one of the six blocks. Both blocks are defined, do not halt internally, and finish at an identical configuration. Their strict interiors contain no modified row, so no nested replacement obligation is being assumed. Iterate.

All block durations are positive and at most four. Thus infinitely many coupled blocks cannot represent a finite run on either side, and a finite halting prefix on either side yields one on the other. The entry-history facts hold independently in both literal tables, including after earlier replacements. This proves both directions without assuming consistency of ZFC or eventual nonhalting. □

The independent checker `code/verify_context.py` reads the two frozen tables directly. It checks the exact delta, all incoming guards, all local traces, arbitrary exterior preservation, strict-interior separation, full state count, canonical dense renaming, and the complete finite blank prefix. It imports neither the allocator nor the symbolic runtime.

2.4 What the engine change buys

A fresh symbolic census of engine execution uses 142 transition rows, compared with 144 for the accepted table. The removed obligations are 31/0 and 42/1; no previously unused engine row is added. This is a statement about engine-phase use, not a statement that these rows are unreachable in the full machine. They still have decoder roles.

The count is established by rederiving every primitive case and separating source-address reading from engine execution at each dispatcher visit. A primitive may revisit the dispatcher internally, so simply treating all rows after the first dispatch as engine rows gives an incorrect census. The included census records use the corrected phase distinction.

Freeing an engine obligation enlarges the allowed decoder allocation. It does not by itself save a working state. Conversely, keeping the old destination of a row that the engine no longer uses would conceal this opportunity from an allocator that freezes whole states.

3 A different 215-state decoder

3.1 One exact source rewrite

Let the accepted source tree be S. At its path

`11010010011101010110011100`

the subtree is $((A, B), C)$. Replace this single occurrence by $(A, (B, C))$. The factors contain 539, 380, and 123 primitive occurrences, respectively. Each has zero escaping break and continuation demand and contains no halt. Thus this is a reassociation of three closed sequential blocks. It changes no primitive opcode, register operand, or control depth.

The emitted source is pinned by SHA-256

`c85089b913dcf446da5039843a43c15129efe88c8ee47a673b5b74600ae42ace.`

Its source graph is checked twice: first by address arithmetic, including the no-stray-suffix condition on each successor, and independently by subtree intervals and ancestor unwinding. Both resolvers produce exactly the accepted ordered effects and guarded successor ordinals, including the initial ordinal and the halt leaf.

Lemma 3.1 (Identity simulation of the source)**.** *For every tuple of nine natural register values, the accepted source and the reassociated source have identical register effects and branch targets at every matching primitive ordinal. In particular they have the same halting behavior from every matching source configuration.*

Proof. Both source graphs are finite and literally equal. At an increment, decrement, break, continue, or halt ordinal, the effect and each guarded target agree. Induction on execution length proves the identity simulation. No bound on a register value is used. □

There remain 202 distinct source subtrees and 4,751 primitive leaves. Exactly 1,042 leaf addresses change, their total address length decreases by 416, and maximum address depth decreases from 47 to 46. Maximum zero run remains 6. These are literal source measurements, not estimates of wall-clock speed or of the time before any possible refutation.

3.2 The frozen physical table

We allocate the rewritten source against the history-return engine, using an engine-row filter and known-prefix checks. The resulting table K_{215}^{shallow} has SHA-256

`7dafe445beb74f66b24d4e540136b5a47d0dfc6c1b877dcb8edce1526a608537.`

It has 215 working states and 430 rows. The allocator changes 191 literal rows relative to the pinned table. Part of this large row delta is decoder relabelling, so the number alone would not establish a meaningful change. The substantive differences are the history-dependent engine returns and the changed source-address tree.

An independent finite graph check proves that the new table is not a rooted, read-labelled, write/move-preserving state renaming of the original. For example, the abstract read-label word `000000` gives different write/move actions in the two rooted transition graphs. This is a graph witness, not a claim

that those reads form a blank-tape execution. A dense canonical table and its complete native-to-dense bijection are supplied.

The allocator's prefix acceptance is only a synthesis result. The physical table is accepted here because of the separate exact runtime and startup checks described next. The proof does not infer binary correctness from the source-node count, the MILP result, or a finite sample of register values.

4 Complete blank starts and quantified binary runtime

Both new tables reach the following complete calibrated frame after 95,740 transitions from a blank tape:

$$q = 84, \qquad \text{head} = -8, \qquad \text{header} = 57, \qquad \text{gap} = 1,$$
$$(N, T, W, A, D, H, X, Y, L) = (4, 0, 0, 0, 0, 0, 0, 0, 0),$$
$$\text{spares} = [1]^{40}.$$

The tape has exactly 95 one cells and is zero everywhere else. This is the same complete frame as the accepted machine, whose prefix takes 95,714 transitions. There are 13 visits to the slower state-13 context in that prefix, accounting exactly for the additional 26 transitions.

The independent frame reconstruction uses the physical order $(W, L, T, D, A, Y, N, H, X)$. There is a one marker at -9, a header one at 57, then unary tokens $0\,1^{v+1}$ for the nine physical register values and forty spare values. This independently gives 95 one cells, with last one at 199. The checker reconstructs this set from the advertised register frame and separately replays every recorded transition; equality of the sets establishes the full zero exterior, not just a displayed tape window.

The source seed has zero formula, stack, and cache registers. Its next candidate value is 4. The inherited calibration expressly permits such a finite cutoff: the proof compiler has cofinally many successful candidate numbers whenever a refutation exists. The frame is therefore calibrated without pretending that the enumerator starts at candidate zero.

Verification quantity	Pinned table	K_{215}^{hist}	K_{215}^{shallow}
Working states	215	215	215
Guarded primitive cases	6,176	6,176	6,176
Symbolic inference steps	1,698,790	1,699,668	1,697,385
Minimum constant duration	6	6	6
Blank-prefix transitions	95,714	95,740	95,740

Table 2: Counts of proof work and exact startup execution. The symbolic inference totals are not TM running times.

The fresh runtime derivations use the repository's archived affine-symbolic checker, with the native table supplied explicitly. Each source increment, break, continue, and halt contributes one guard case; each decrement has zero and positive cases. The 4,751 occurrences therefore give 6,176 cases. Each derivation quantifies over all natural primary register values consistent with its guard, an arbitrary natural gap, and a nonempty arbitrary finite spare list, including an arbitrary first spare and tail.

The proof rules consist of literal table transitions, induction over unary scan lengths, and four separately derived list-block schemas. There is no fixed maximum register value, proof length, or spare-list length in those claims. The complete normalized frame and zero exterior are part of the primitive contracts. Every reported affine duration has a positive constant term and nonnegative variable coefficients; the minimum constant is 6. Thus an unbounded source run corresponds to an unbounded binary run.

The exact table and source hashes, the checker and helper hashes, case identities, and failures list are recorded in the runtime receipts. All cases succeed. The independent shallow-decoder audit additionally checks the complete unique guard coverage and binds the receipt to the included runtime code. The compressed case lists and startup traces are included without discarding difficult or exceptional entries.

These are ordinary mathematical derivations carried out by executable checkers and independently reviewed arguments. They are not newly compiled Lean or Rocq proofs of these exact tables. The packet does not claim a proof-assistant kernel result, a PA endpoint, or an unconditional proof of nonhalting.

5 The resulting consistency equivalences

Theorem 5.1 (The supplied 215-state machines)**.** *For each* $M \in \{K_{215}^{\mathrm{hist}}, K_{215}^{\mathrm{shallow}}\}$*, ordinary reasoning formalizable in ZFC establishes*

$$\begin{aligned} \mathrm{Halt}(M, \mathsf{blank}) &\longleftrightarrow \neg\,\mathrm{Con}(\mathsf{ZFC}), \\ \neg\,\mathrm{Halt}(M, \mathsf{blank}) &\longleftrightarrow \mathrm{Con}(\mathsf{ZFC}). \end{aligned}$$

Here consistency is the same arithmetized absence of a finite ZFC refutation used in the pinned source calibration.

Proof. For K_{215}^{hist}, Section 2 proves blank halting equivalence to the accepted exact table. Compose with that table's calibrated theorem [1, 2, 3]. The fresh runtime and blank checks supply an additional direct audit of the same implementation.

For $K_{215}^{\mathrm{shallow}}$, the independent blank replay reaches the calibrated source root. The quantified primitive-runtime theorem relates every finite source step to a finite positive-duration binary block. The equal source graph of Section 3 identifies that source with the calibrated accepted source on all register values. Its command soundness invariant, complete proof-block compiler, cofinal padding, and strictly draining candidate reader yield halting exactly when the dominated numerical calculus proves its falsity code. The inherited calibration identifies that event with a finite ZFC refutation. The reverse simulation and positive durations exclude an extra binary infinite behavior inside a source primitive. The finite initial prefix then gives the blank-input statement. □

The consistency statement is not assumed in this proof of an equivalence. If ZFC is consistent, the theorem has the expected unprovability consequence; it does not license claiming that ZFC proves its own consistency. All mathematical results needed beyond the new finite machine and source arguments are identified in the cited repository calibration.

6 A 201-subtree source: transfer-pop and cache-return inference

This construction combines the transfer-pop proof enumerator of [6] with cache-return modus ponens from [5], rebased onto the accepted K215 source [1]. The combination needs a new proof compiler: cache-return inference no longer supplies the dummy stack entry consumed when its result is transferred to the current formula. Reserving that entry before compiling the premises resolves the mismatch. The resulting literal programs have 201 distinct source subtrees, one fewer than the accepted source. This section proves their ordinary source-level consistency calibration. It does not identify a 214-state binary machine or infer a physical state count from the source count; allocation, full-tape simulation, and blank initialization are separate obligations.

6.1 Numerical calculus and exact command interface

Use Cantor pairing and the unchanged implication and quantifier codes:

$$\mathsf{P}(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad \mathsf{I}(a,b) = 4\mathsf{P}(a,b), \qquad \mathsf{Q}_i a = 4\mathsf{P}(i,a) + 2.$$

Truth is code0 and falsity is code1. In particular, $\mathsf{I}(0,0) = 0$. Write $[x;w] = \mathsf{P}(x,w)$ and use right-nested pairing for longer stacks. This is numerical notation, not an injective encoding of finite lists: $[0;0] = 0$, and both projections of zero are zero.

Let $\mathcal{D}$ be the accepted dominated numerical calculus: truth, numerical MP, theorem-only generalization, the seven retained logical rows $R_1, \ldots, R_7$, the complete gated theory bundle, and

$$L(A,B,C,U) = \mathsf{I}\big(\mathsf{I}(\mathsf{I}(C,U),A), \mathsf{I}(\mathsf{I}(A,C), \mathsf{I}(B,C))\big), \qquad U \le A.$$

These rows and the full bundle are retained literally. In particular, none of the bundle's support or atomic-vacuity material is removed. The inherited finite propositional certificate recovers unrestricted ρ_8 for every natural code; the accepted calibration then gives

$$\mathcal{D} \vdash 1 \quad \Longleftrightarrow \quad \exists p \ \mathrm{Proof}_{\mathrm{ZFC}}(p, \bot). \tag{2}$$

The present theorem uses this existing mathematical calibration [1]; it is not a new proof-assistant verification of that dependency. Formula decoding and well-formedness tests are not added to the numerical calculus.

The proof enumerator has a new operational language with unchanged axiom strength. In particular, the operand of MP remains an arbitrary natural increment; inference is not restricted to a fixed consequent or a fixed parameter. Successful inference, cache movement, and stack restoration are reorganized together. Treating either command change in isolation would miss the obstruction solved by the reserved zero, and would not justify transferring the old completeness theorem.

At command boundaries the work tuple is (t,w,d,h): current formula, paired stack, additive parameter, and cached formula. If $\mathrm{unpair}(w) = (x,v)$, the direct transfer command is

$$\mathsf{S}(t,w,d,h) = \begin{cases} \mathsf{halt}, & h = 1, \\ (h,v,d,x), & h \neq 1. \end{cases}$$

For $Y = d + x$, cache-return inference has the exact equation

$$\mathsf{MP}(x)(t,w,d,h) = \begin{cases} (0,w,0,Y), & \mathsf{I}(t,Y) = h, \\ (0,w,Y,0), & \mathsf{I}(t,Y) \neq h. \end{cases}$$

The successful branch clears the parameter and leaves the conclusion in cache; it does not push a dummy zero. The failed branch is unchanged.

The axiom command $\mathsf{AX}(c;x,y)$ is unchanged. Set $U = d$, $A = d + x$, $B = h + y$, and $C = c$. Its result is

$$t' = G(A,B,c), \qquad d' = 0, \qquad h' = 0,$$
$$w' = [R_1, \ldots, R_7, L(A,B,c,U), \mathsf{Q}_U h, t; w].$$

Here G is the original full bundle when $c \neq A, B$, and zero otherwise. The quantified entry uses the incoming cache, before adding y. Every logical row is an implication code and hence differs from1; the quantified entry is nonzero of residue2 modulo4. The gate is also never1: an enabled bundle is implication-coded, and a disabled gate is zero. These facts hold for arbitrary raw operands.

6.2 Literal construction and eight variants

In the accepted tree, replace the successful-MP subtree at `11010010111` by $(\mathrm{CD}_4, (\mathrm{CI}_5, \mathrm{CC}_2))$. Delete the DROP selector layer by replacing `11010` with its current descendant `11010010`. Finally replace the old NIP body at `111` by a transfer body. The direct body is the closed composition

$$\mathsf{clearT}; \mathsf{copyH}; \mathsf{oldNIP}.$$

The first part clears current; the second is the old DROP cache-copy tail, preserving cache and clearing scratch. Old NIP then tests that cache and pops the stack. Both parenthesizations implement precisely the direct S equation.

Alternatively use $\mathsf{oldDROP}; \mathsf{oldNIP}; \mathsf{oldNIP}$, abbreviated DNN. Its nonhalting map is the same, but it also halts if incoming $t = 1$. It is therefore an early-halting refinement of direct transfer. Each body has two parenthesizations, and each program has two small selector orders, giving eight literal variants:

Transfer body	Variants	Distinct subtrees	Leaves	Depth	Zero run
Direct	4	201	4709	44	6
DNN	4	201	4752	44	6

For the ordinary order, wire fields are

$$f(\mathsf{S}) = [0], \qquad f(\mathsf{MP}(x)) = [1, x], \qquad f(\mathsf{AX}(c; x, y)) = [c + 2, y, x].$$

The `swap01` order exchanges only the first two selectors: $f(\mathsf{S}) = [1]$ and $f(\mathsf{MP}(x)) = [0, x]$. AX fields stay fixed. The first two selector decrements prove these partitions for all natural fields: zero selects the first body, one the second, and $c + 2$ enters AX with residual c.

The independent source audit reconstructs all eight files, checks both CFG resolvers, and verifies all 4565 primitive effects and internal successors of AX. Its unique exterior edge still reaches the common branch continuation. Thus AX is not incorrectly assumed closed when its required outer continuation is supplied by the selector.

6.3 Soundness on all raw candidate words

Call a work tuple safe if current, cache, and every successive decoded stack head are $\mathcal{D}$-theorems. No theoremhood condition is imposed on d. AX preserves safety by its inherited axiom and generalization contracts. Failed MP inserts truth; successful MP derives Y from t and $\mathsf{I}(t, Y)$, places it in cache, and inserts truth in current. Transfer only moves theorem codes. Its cache-one guard, or either guard of DNN, therefore witnesses a theorem equal to falsity. Consequently every actual halt from a safe seed yields $\mathcal{D} \vdash 1$. This argument quantifies over all natural command fields, including unintended or nondecodable inputs; it does not presuppose a valid compiled proof word.

6.4 Structural blocks recovered without DROP

Put $D_0 = \mathsf{AX}(0; 0, 0)$ and define

$$\begin{aligned} \mathsf{H} &= D_0; \mathsf{S}^{11}, \\ \mathsf{PUSH} &= D_0; \mathsf{S}^{9}; \mathsf{MP}(0), \\ \mathsf{DISCARD} &= \mathsf{MP}(1); \mathsf{AX}(1; 0, 0); \mathsf{S}^{10}; \mathsf{MP}(0). \end{aligned}$$

From $(t, [x; w], d, h)$, the first ten shifts of H yield $(\mathsf{Q}_d h, [x; w], 0, t)$. Their tests inspect only zero and the nine generated nonfalsity entries. The eleventh shift halts exactly when $t = 1$, otherwise returning $(t, w, 0, x)$. Thus H preserves current while popping one old stack entry and resetting the parameter. It intentionally does not test the original cache before AX erases it.

After nine shifts in PUSH, current is the dominated row, cache is $\mathsf{Q}_d h$, and stack is $[t; w]$. The final MP necessarily fails because an implication code has residue0 and this cache has residue2. Hence PUSH never halts and returns $(0, [t; w], 0, 0)$ for every natural input tuple.

DISCARD starts from $(t, w, 0, 0)$. Its first MP fails since $\mathsf{I}(t, 1) > 0$. AX has $U = A = c = 1$ and gate zero. Ten shifts give $(\mathsf{Q}_1 0, w, 0, 0)$; the final MP fails because $\mathsf{I}(\mathsf{Q}_1 0, 0) > 0$. Thus DISCARD never halts and returns $(0, w, 0, 0)$. Finally, from current, cache, and parameter zero, define $J(0)$ to be empty and $J(u) = \mathsf{MP}(u)$ when $u > 0$. The latter is a forced failure setting $d = u$. Omitting $\mathsf{MP}(0)$ is essential: the zero alias would make it succeed.

For DNN, the extra top test does not invalidate these exact auxiliary claims: every tested gate and generated entry is nonfalsity. The old top remains untested in PUSH, and its sole test in H is the intended last shift.

These blocks use the axiom emitter as a source of temporary storage. Its fixed ten-entry output preserves the old current formula behind nine safe entries; repeated transfers can recover that saved value or leave it on the stack without a primitive DROP command. The intermediate logical rows need not be useful premises in the proof being compiled. Their exact stack positions, their nonfalsity tags, and the quantified entry's residue are sufficient. This distinction matters for compression: a command that is expensive as an isolated arithmetic routine can supply several structural operations through already shared source subtrees. The resulting proof words can be very long. No execution-time advantage is required for a consistency machine, whose specification asks only whether a finite halting run exists.

6.5 The strong theorem-block compiler

For every derivable code X, construct a finite block $\mathcal{B}_X$ which, from *every* $(t, w, 0, 0)$, either halts or returns exactly $(X, w, 0, 0)$. Arbitrary incoming t, w need not be safe. This universal completeness contract is distinct from the safe-run soundness invariant. Truth uses DISCARD. The axiom cases are

$$\begin{aligned} R_j: &\quad \mathsf{DISCARD}; \mathsf{AX}(c; a, b); \mathsf{S}^{j+1}; \mathsf{H}^{9-j} && (1 \le j \le 7), \\ L(a, b, c, u): &\quad \mathsf{DISCARD}; J(u); \mathsf{AX}(c; a - u, b); \mathsf{S}^9; \mathsf{H} && (u \le a), \\ G(a, b, c): &\quad \mathsf{DISCARD}; \mathsf{AX}(c; a, b); \mathsf{H}^{10} && (c \ne a, b). \end{aligned}$$

The indicated shifts select the desired row; the H repetitions preserve it and remove exactly the remaining generated entries, ending with saved zero in cache. The underlying stack is restored. All auxiliary MP calls are forced failures, so cache-return success does not affect these calculations.

Generalization uses

$$\mathcal{B}_{\mathsf{Q}_i X} = \mathcal{B}_X; \mathsf{PUSH}; J(i); \mathsf{S}; \mathsf{AX}(i; 0, 0); \mathsf{S}^{10}.$$

After PUSH, setter, and transfer, the state is $(0, w, i, X)$. That transfer tests incoming cache zero, not the newly popped X. AX has zero gate and generates $\mathsf{Q}_i X$ immediately above saved zero; ten shifts return $(\mathsf{Q}_i X, w, 0, 0)$. This includes $i = 0$ and $X = 1$, subject only to permitted earlier halting.

MP with a protected zero. For premises A and $\mathsf{I}(A, Y)$ define

$$\mathcal{B}_Y = \mathsf{DISCARD}; \mathsf{PUSH}; \mathcal{B}_{\mathsf{I}(A,Y)}; \mathsf{PUSH}; \mathcal{B}_A; \mathsf{PUSH}; \mathsf{S}; \mathsf{S}; \mathsf{MP}(Y); \mathsf{S}.$$

If no earlier halt occurs, its exact state trace is

$$\begin{aligned}(t, w, 0, 0) &\longmapsto (0, [0; w], 0, 0)\\ &\longmapsto (0, [\mathsf{I}(A, Y), 0; w], 0, 0)\\ &\longmapsto (0, [A, \mathsf{I}(A, Y), 0; w], 0, 0)\\ &\longmapsto (0, [\mathsf{I}(A, Y), 0; w], 0, A)\\ &\longmapsto (A, [0; w], 0, \mathsf{I}(A, Y))\\ &\longmapsto (0, [0; w], 0, Y)\\ &\longmapsto (Y, w, 0, 0).\end{aligned}$$

The second transfer may halt if $A = 1$, and the final transfer halts if $Y = 1$. Otherwise the displayed equations hold. Premise blocks preserve their entire incoming stacks, which protects the reserved zero until it compensates for the final pop. Without this reservation, that pop would destroy an arbitrary original stack head. Pairing identities validate the trace even at $A = Y = w = 0$; no list freeness is assumed.

Induction on a finite derivation proves the strong contract. A proof DAG can be expanded to a finite tree before compiling the major premise first. DNN transfers the contract unchanged: runs agree with the direct version until DNN possibly halts earlier. If $\mathcal{B}_1$ returns, append H to detect its current falsity. Thus $\mathcal{B}_1; \mathsf{H}$ succeeds from every $(t, w, 0, 0)$.

6.6 Cofinal successful candidates and the source theorem

Let $F(fs, r)$ right-fold the command fields above residual r, and set

$$\mathrm{tail}([]) = 0, \qquad \mathrm{tail}(k :: L) = 1 + F(f(k), \mathrm{tail}(L)).$$

A continuation first decrements its positive proof-list pointer; a fresh candidate decodes without that outer successor. Put $R = \mathrm{tail}(\mathcal{B}_1; \mathsf{H})$. For each natural z,

$$N_z = \mathsf{P}\big(2, \mathsf{P}(z, \mathsf{P}(1, R))\big)$$

starts with $\mathsf{AX}(0; 1, z)$ in either selector order. This resets $d = h = 0$ from every retained tuple. Its current and stack may be arbitrary, precisely as allowed by the strong contract. Since $N_z \geq z$, successful candidates are cofinal beyond every fixed finite startup cutoff.

Every nonhalting positive continuation strictly decreases: after its predecessor operation, each field reader takes a Cantor right projection no larger than that predecessor. The natural-counter macros terminate. Hence each nonhalting candidate drains, and sequential enumeration reaches a successful N_z unless it has already halted.

Theorem 6.1 (Source consistency calibration)**.** *For each of the eight literal transfer/cache programs, started with the unchanged enumerator scheduler and a safe initialized frame, halting is equivalent to* $\mathcal{D} \vdash 1$. *Consequently, using* (2)*, its nonhalting is equivalent in ZFC to* Con(ZFC).

Proof. Safe-run soundness gives the forward implication. A native refutation gives the strong block and cofinally many successful candidates, proving the reverse implication. These finite syntactic transformations and natural-number arguments are formalizable in ZFC. A binary realization requires its own exact runtime and initializer proofs. □

Open parity-alarm experiment. A separate 205-subtree, 5025-leaf source replaces emitted equality $E(a,b) = 4\mathsf{P}(a,b)+3$ by $\mathsf{I}(E(a,b),1)$, interprets raw residue3 atoms as disequality, and tests cache parity with the four-leaf scanner $(\mathrm{CD}_5,((\mathrm{CD}_5,\mathrm{CC}_2),\mathrm{CH}))$. Constant assignments in an irreflexive model make both odd atom classes false. However, the false witness $1 = \mathrm{Rel}(0,0)$ adds syntactic variable0 to the equality representation. Semantic validity alone does not establish support-sensitive vacuity or the exact formula/binder coupling in the gated bundle. A revised encoder and reverse calibration remain to be proved; this experiment is not covered by the theorem above.

7 A semantic alarm: every membership atom detects inconsistency

The halting test need not recognize one distinguished contradiction code. It may recognize any decidable family of codes that contains that contradiction and contains no theorem under the consistency hypothesis. This observation separates the semantic choice of a halting predicate from the cost of implementing it. The following application changes the alarm from equality with 1 to the congruence $h \equiv 1 \pmod 4$. It is an alternative source-level construction; the physical allocation reported below does not produce a new 215-state machine.

7.1 A general replacement principle

Let $\mathcal{D}$ be the calibrated numerical calculus, with falsity code f, and let E be its refutation enumerator. At command boundaries, assume the usual invariant: the current formula, the cache, and every decoded stack head are $\mathcal{D}$-theorems. The original alarm halts when the cache is f. Its other commands preserve this invariant, and its proof compiler and scheduler establish refutation completeness.

Proposition 7.1 (Semantic alarm replacement)**.** *Let $A \subseteq \mathbb{N}$ be decidable, with $f \in A$. Suppose ZFC proves*

$$\mathrm{Con(ZFC)} \implies \forall a \in A\ \neg(\mathcal{D} \vdash a).$$

Replace the original alarm by membership in A, preserving its complete nonhalting command effect. From any calibrated seed, the modified enumerator halts exactly when ZFC is syntactically inconsistent.

Proof. Under Con(ZFC), the invariant excludes an alarm value at every command boundary. Each nonhalting modified command has the old invariant-preserving effect, so the new enumerator cannot halt.

For the converse, compare the two enumerators from the same raw boundary state. Until the new one halts, their boundary stores coincide. When the old alarm would halt, its cache is $f \in A$, so the new alarm also halts, unless it has already halted earlier. Induction on a finite old halting execution proves that every such execution is detected by the new enumerator. Applying the old refutation-completeness theorem gives the result. This comparison does not assume that the starting store is safe: its only premise is equality of the nonhalting command effects. □

A sufficient semantic condition is that, assuming consistency, each $a \in A$ has a counterassignment in a model in which numerical derivations are sound. The model and assignment may depend on a. The application below uses one nonempty model and one constant assignment to refute every member of the alarm family simultaneously.

7.2 All raw codes of residue one

Write

$$P(i,j) = \frac{(i+j)(i+j+1)}{2} + j, \qquad R(i,j) = 4P(i,j) + 1.$$

The total numerical decoder used in the inherited calibration satisfies $\mathrm{dec}(R(i,j)) = \mathrm{Rel}(i,j)$. Its membership and soundness lemmas are `dec_wel`, `sat_wel`, and `c254_sound` in `Code254.lean`.[4] Cantor unpairing gives, for every natural h,

$$h \equiv 1 \pmod 4 \quad \Longleftrightarrow \quad \exists i, j \in \mathbb{N}\ h = R(i,j).$$

Indeed, unpairing $(h-1)/4$ supplies the required indices. Thus every accepted raw number is a membership atom. No restriction to numbers emitted by a proof compiler, no freshness test, and no separate well-formedness test is present. The truth alias 0 is excluded.

Lemma 7.2. *In a nonempty irreflexive native model satisfying the calibrated bundles, no membership-atom code is derivable in $\mathcal{D}$.*

Proof. Let the model have domain V and relation $\in_M$, with $\neg(v \in_M v)$ for every $v \in V$. Choose $v \in V$ and put $e(k) = v$ for every natural variable index k. If $\mathcal{D} \vdash R(i,j)$, code soundness makes its decoded formula true under every assignment, hence under e. The literal atom clause then gives $v \in_M v$, a contradiction. The argument applies also when $i \neq j$: distinct variable names may receive the same value. The dominated calculus is contained in the calibrated unrestricted calculus, so its soundness follows directly; the stronger theorem-set equality established for dominated ρ_8 is not needed in this direction.[3] □

The model-existence step is conditional and takes place within ZFC. From the syntactic statement Con(ZFC), the first-order completeness theorem supplies a set-coded model of ZFC. The inherited native/ZFC interpretation then supplies a nonempty native model satisfying the exact gated bundles. Its membership relation is irreflexive, as required by the calibrated semantics.[1, 4] No transitive model and no externally well-founded model is assumed. Foundation is used through what the model satisfies, not through an external well-foundedness claim about its relation. Consequently ZFC proves

$$\mathrm{Con(ZFC)} \Longrightarrow \forall h \in \mathbb{N}\,(\mathcal{D} \vdash h \Longrightarrow h \not\equiv 1 \pmod 4).$$

This is precisely the soundness premise of Proposition 7.1, with $f = 1$.

The actual-run argument covers unintended command encodings as well as compiled proof words: the old invariant says that the cache is a theorem at every alarm entry. Conversely, $1 \equiv 1 \pmod 4$, so every old refutation is still detected. The strong compiler's "halt or return exactly" contracts, its cofinal padding above arbitrary finite cutoffs, and the decreasing continuation rank therefore transfer unchanged for completeness.[1] Expanding the alarm further to all odd numbers would be unsound for this encoding: the equality code $4P(i,i)+3$ is derivable and has odd parity. The distinction between the two atomic tags is essential.

7.3 Exact scanner and measured cost

The residue test can destructively scan the cache with a four-phase cycle. Each phase tests for zero and otherwise decrements it. The zero exits of phases $0, 1, 2, 3$ are respectively

$$\text{return}, \qquad \text{halt}, \qquad \text{return}, \qquad \text{return}.$$

Four positive decrements return to phase 0. Induction on the quotient in $h = 4q + r$ proves the exact behavior for every natural cache value, and every normal exit has cache zero. Appending the unchanged stack-pop tail gives precisely the old NIP nonhalting output, including its scratch-register conditions.

The supplied search exhausts a declared finite family of seven-leaf structured scanners: four cache decrements, one halt, one backward continue, and one forward break, with closed control flow. It checks their resolved phase graphs rather than testing finitely many cache values. Of 28,452 specifications, three have the required graph. The best resulting complete source has 206 distinct subtrees, 4,746 primitive occurrences, maximum depth 47, and maximum zero run 6. The exact source is `sites-atomhalt-best.json`; its search record and reconstruction program accompany it.

The attempted 215-state allocation was infeasible in the declared protected-row allocation model. This is neither a lower bound for the formalism nor a claim of a 206-state Turing machine: 206 counts source subtrees. No accepted 214- or 215-state binary implementation of this alarm is supplied. A future implementation must still prove its all-natural primitive runtime and reach a calibrated blank-tape seed. The semantic theorem remains independent of that unresolved encoding cost, and no global priority claim is made for the replacement principle.

8 What the searches establish, and what remains open

The source and physical searches were run against the pinned 215 table, not against a stale larger candidate. Their useful output is both the explicit alternatives above and a more specific account of why several attractive source reductions do not immediately lower the binary count.

8.1 Source size and physical sharing are different objectives

The successful-MP implementation can be shortened in several ways. Returning the result in the cache or omitting the dummy stack push each gives a 202-subtree source at this stage. Combining cache return with the three-command transfer system gives 201 subtrees. Nevertheless, ordinary row-protected allocation at 215 states failed for the tested direct, reassociated, and opcode-swapped versions. It also failed with the two new engine freedoms from Section 2.

One concrete reason is that an old successful-MP subprogram participates in a context-sensitive marker fold. Removing its useful semantic work can also remove its physical sharing opportunity. For the 201-subtree design, the global allocation still needs 133 newly assigned decoder roles plus 32 fixed physical decoder states. Matching this total to the old total does not guarantee compatibility with the surviving engine rows and helper destinations.

A diagnostic relaxation identifies particular obstructions: allowing the allocator to overwrite any one of the protected rows 26/0, 26/1, 37/0, or 302/0 permits a 215-state prefix allocation of the direct 201-subtree source. These relaxed outputs are *not accepted machines*; they would alter engine execution without a proof. Moving the engine's 37/0 return to another existing provider did not by itself resolve the entire allocation. This narrows a future attack to a concrete selector interface instead of suggesting that one missing source node is automatically one missing binary state.

8.2 Finite search boundaries

The recorded searches include 287 global source-reassociation candidates, of which 216 passed the exact graph condition. Thirty-one distinct 202-subtree ties were saved; none in that family had 201 subtrees. A cache-return-specific search considered 729 local reassociations with structured-control-depth repair

and obtained 137 valid graph candidates, again without a 201-subtree improvement of that source. These are bounded families of syntax transformations, not lower bounds over all source programs.

On the physical side, 56,658 existing-helper context-fold specifications were checked on the original table and 58,788 on the history-return table, with no additional accepted fold. Another 26,896 ordered direct-context contractions had no survivor. A local retargeting search considered 274,631 specifications and supplied the exact history-dependent blocks used here. Its failure to find a state deletion does not imply that every possible multirow redesign is impossible.

A final search synthesized both rows of an existing helper while contracting a decoder pair. It emitted 336 structural 214-state trials; none passed the exact symbolic block/primitive derivation filter. These rejected trials are not candidate solutions. Separately, freeing all admissible singleton nonpure decoder roles simultaneously gave a valid 215-state control allocation, but still no allocation for the direct 201-subtree source. The recorded negative results remain relative to their specified synthesis and proof-rule families.

The membership-alarm source search enumerated 28,452 structured scanner specifications with four cache decrements and one each of halt, continue, and break. It accepted three exact four-phase scanner graphs. Its best complete source had 206 distinct subtrees. The acceptance condition was an exact finite control graph establishing the all-natural residue behavior, not testing the first several hundred cache values.

There is also an exact zero/positive abstract-state analysis of the accepted source. It identifies the always-zero decrement at the beginning of the successful-MP dummy push and two consequent unreachable leaves. Deleting that empty transfer alone produces 203 source subtrees: duplicated surrounding structure offsets the local deletion. This is another example of why raw leaf savings can be misleading.

8.3 The next concrete targets

The strongest uncompleted target is the three-command cache-return source. It has a complete source-level consistency compiler and fewer distinct subprograms, but its engine/decoder overlap still needs a successful exact allocation. A useful next experiment is a multirow redesign of the state-26 selector, paired with allocation of its old decoder role; a local engine return that is correct only after entry through state 11 must be checked under that specific entry contract and then subjected to the full runtime checker.

The other target is a cheaper code family for a semantic alarm. The membership class demonstrates that the alarm need not be a distinguished literal falsity number. The challenge is to make the safe class cheap to recognize without increasing the cost of all formula constructors or silently breaking the source's numerical binder and support conditions. The separate negative-equality parity note marks precisely that remaining calibration obligation.

The two accepted machines in this packet are implementation alternatives at 215 states. The 201-subtree calculus and semantic alarm are substantive formalism directions. Neither is described as an achieved 214-state breakthrough, and no exhaustive optimality or global novelty claim is made. Existing cache-return and transfer-pop ideas are credited to the integrated repository reports [5, 6]. The newest incoming binary ZIP submissions were not available as readable source through the text retrieval interface, so priority comparisons are limited to the integrated material actually inspected.

9 Reproduction and incorporation

Run the following from the extracted packet directory with Python 3.10 or later. The finite checks and the symbolic runtime use the standard library. The allocator alone needs NumPy and SciPy and is not needed to verify the frozen machines.

```
python3 code/verify_context.py
python3 code/audit_shallow.py
python3 code/audit_runtime_receipts.py
python3 research/verify_cache_transfer.py
python3 research/build_cache_transfer.py
```

The last command also reruns explicitly labelled finite command regressions. The separate all-natural command proof is in this report and `research/cache_transfer_proof.md`.

To rederive the unbounded primitive contracts rather than only checking their archived receipts:

```
python3 code/runtime-inputs/validate_native.py \
  machine/k215-context.native.tm machine/sites215.json \
  reproduction/context-runtime

python3 code/runtime-inputs/validate_native.py \
  machine/k215-shallow.native.tm machine/sites-shallow.json \
  reproduction/shallow-runtime
```

The exact checker code is vendored with its inherited notices. These runs write complete startup traces and per-guard case records, so their output is larger than a short success log. A failure to derive a case would be a failed certificate, not automatically a counterexample to the machine's semantics.

Compile the paper with `pdflatex report.tex` twice, or use the included `Makefile`. The dense full transition table in the appendix is generated from the frozen native shallow-decoder table by `code/audit_runtime_receipts.py`. Thus the PDF also contains a complete machine specification; the machine-readable tables remain the preferred input for independent work.

The packet is intended as an additive research contribution. Inherited tables, utilities, checker code, mathematical calibration, notices, and attributions retain their existing identity and terms. The root README states the exact new claims, source-only results, proof status, and file layout. The archive contains no claim that the new exact tables have been compiled through a proof-assistant kernel.

References

[1] ProveIt contributors, *Ordinary proof of the combined215 machine*, `Computability/BusyBeaver/Research/ZFC/candidate-k215/proof.md`, commit `75716498437bfc7291d5900cab2983a11fd54bac`. Pinned source.

[2] ProveIt contributors, *All-natural calibration of the permuted command source*, candidate-k215 provenance, `permuted-commands/sections/source_calibration.tex`, same commit. A copy is included as `upstream/source_calibration.tex`.

[3] ProveIt contributors, *Dominated numerical-calculus proof*, `Computability/BusyBeaver/Research/ZFC/candidate-k218/alternatives/dominated/proof.md`, same commit. Pinned source.

[4] ProveIt contributors, *Code254.lean*, `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/Code254.lean`, same commit; in particular the literal membership constructor, its decoding/satisfaction lemmas, and code soundness. Pinned source.

[5] ProveIt contributors, *Cache-return modus-ponens proof*, integrated report `54-cachemp-proofs-cache_mp_proof.md`, `SetTheory/Cardinals/docs/reports/set-theory/zfc-closure-kernels/`, same commit. The cache-return interface predates the present composition.

[6] ProveIt contributors, *Complete proof compiler for the three-command NIP-transfer enumerator*, integrated report `166-transfer-proofs-transfer_compiler.md`, in the same report directory and commit; see also its composite-transfer review.

A Complete dense transition table for the shallower 215-state machine

Start in state 0 on an all-zero two-way tape. The working states are $0, \ldots, 214$; the only terminal state is 215. An entry wDz means write w, move in direction D, and enter z. Each line below gives both read cases for three working states. The final empty right block is only table layout, not a missing transition.

State	Read 0	Read 1	State	Read 0	Read 1	State	Read 0	Read 1
0	1R1	1R0	72	0R89	1R90	144	0R159	1R160
1	0R2	0R0	73	1R73	1L13	145	0L161	1R162
2	1R3	1L4	74	0R51	1R74	146	0R46	1L137
3	1L4	1L5	75	0L73	0R91	147	0R98	1R163
4	0L6	1L4	76	0R92	1R76	148	0R145	1R164
5	0L7	0L8	77	0R46	1R93	149	0R111	1R52
6	0L9	1L4	78	0R94	1R68	150	0R80	1R72
7	0L9	0L7	79	0R95	1R96	151	0L164	1R105
8	1L5	1L8	80	0R97	1R98	152	0L37	0R50
9	1R10	0L9	81	0R44	1R99	153	0R89	1R121
10	0R10	0R11	82	0L100	1L101	154	0R155	0L12
11	0L12	0R11	83	0R102	1R103	155	0L53	1R165
12	1L13	0L13	84	0R104	1R105	156	0R166	0L12
13	0L13	1L14	85	0R106	1R107	157	0R105	1R78
14	0L15	1L14	86	0R108	1R109	158	0R98	1L103
15	0L16	1L14	87	0R75	0R92	159	0R78	1R83
16	0L17	1L14	88	0R110	1R111	160	0R167	1R168
17	0L18	1L14	89	0R57	0R92	161	0R126	1R169
18	0L19	1L14	90	0R41	0L12	162	0R78	1R104
19	0L20	1L14	91	0R112	1R91	163	0R80	1R125
20	0L21	1L14	92	0R38	1R92	164	0R170	1R133
21	0L22	1L14	93	0R113	0L7	165	0R53	1L40
22	0R22	1R23	94	0R72	1R114	166	0R123	1R123
23	0R24	1R25	95	0R115	1R44	167	0R84	1R158
24	0R26	1R27	96	0R116	1R117	168	0R171	1R158
25	0R28	1R29	97	0R33	1R118	169	0R151	1R172
26	0R30	0L7	98	0R119	1R120	170	0R158	1R173
27	0R31	1R32	99	0R121	0L12	171	0R104	1R78
28	0R33	1R34	100	0R104	1R122	172	0R102	1R174
29	0R35	1R36	101	0R123	1R124	173	0R175	1R176
30	0L37	0R38	102	0R125	1R126	174	0R177	1R104
31	0R39	1R40	103	0R127	1R128	175	0R98	1R178
32	0R39	1R41	104	0R94	1R129	176	0R179	1R180
33	0R42	1R43	105	0R94	1R130	177	0R82	1R181
34	0R44	1R45	106	0R131	1R132	178	0R167	1R182
35	0R46	1R47	107	0R133	0L7	179	0R183	1R181
36	0R41	1R48	108	0R57	0R74	180	0R184	1R185
37	0L29	1R49	109	0R87	1R56	181	0L186	1R187
38	0R50	1R38	110	0R44	0L11	182	0R123	1R168
39	0L37	0R51	111	0R56	1R90	183	0R188	1R189
40	0R30	1L52	112	1R0	1R91	184	0L190	1L83
41	0L53	1L54	113	0R37	1R134	185	0R181	1R146
42	0R30	1R55	114	0R135	1R136	186	0R184	1R191
43	0R56	1R40	115	0R108	1R117	187	0R82	1R98
44	0R57	0R58	116	0L137	0L13	188	0R192	1R161
45	0R59	0L12	117	0R110	0L7	189	0R190	1R193

Continued on the next page

State	Read 0	Read 1	State	Read 0	Read 1	State	Read 0	Read 1
46	0R57	0R60	118	0R138	1R105	190	0R175	1R82
47	0R61	0L7	119	0R139	1R140	191	0R184	1R194
48	0R62	1R63	120	0R141	1R142	192	0R123	1R184
49	0R64	1R65	121	0L42	1L86	193	0R127	1R195
50	0R12	1R50	122	0R143	1R127	194	0R196	1R197
51	0R66	1R51	123	0R87	1R87	195	0R198	1R186
52	0R67	1L68	124	0R144	1R145	196	0R187	1R181
53	0R69	1R54	125	0R105	1R104	197	0R199	1R196
54	0L28	1R70	126	0R98	1R123	198	0R200	1R201
55	0R69	1R71	127	0R126	1R98	199	0R184	1R202
56	0L72	1L55	128	0R64	1R87	200	0R101	1R203
57	0L73	0R0	129	0R146	1R34	201	0R143	1R204
58	0R74	1R58	130	0R52	1R138	202	0R205	1R205
59	0R75	0R60	131	0R63	1R147	203	0R206	1R177
60	0R76	1R60	132	0R148	1R149	204	0R207	1R208
61	0R77	1R78	133	0R79	1R116	205	0R162	1L209
62	0R79	1R215	134	0R150	1R151	206	0R184	1R210
63	0L80	1R81	135	0R152	1R70	207	0R100	1R211
64	0L82	1L83	136	0R141	1R140	208	0R212	1R197
65	0R84	1R85	137	0R86	1R153	209	0R145	1R168
66	0R60	1R66	138	0R44	1R154	210	0R64	1R213
67	0R57	0R76	139	0R152	1L54	211	0R171	1R205
68	0R86	1R81	140	0R56	1R139	212	0R125	1R214
69	0R75	0R66	141	0R153	1R155	213	0R102	1R64
70	0R87	0L12	142	0R56	1R156	214	0R125	1R160
71	0R88	0L70	143	0R157	1R158			